

Problem & Motivation

- LLM decoding is **inherently sequential**: generating n tokens requires n forward passes through the full target model, making latency grow linearly with output length.
- Speculative decoding** uses a lightweight *draft model* to propose K candidates; the target verifies all in one batched forward pass. Higher-quality drafts yield more accepted tokens per step.
- Our contribution**: Port DFlash—a diffusion-based block drafter—from GPU/PyTorch to TPU/JAX inside **tpu-inference**. Benchmark Qwen3-4B across 9 datasets (math, code, chat).
- Why TPU?** Verification cost is **flat from $K=16$ to $K=128$** ($0.97\times$), while GPU scales to $2.3\times$ —ideal for aggressive speculative decoding with wide draft blocks.

Background

Speculative Decoding

- A fast draft model generates K candidates; the target scores all K in one pass via rejection sampling. Accepted tokens are emitted *without recomputation*.
- Key metric: τ = average accepted tokens per draft step. Ideal $\tau \rightarrow K$; limited by draft–target distribution mismatch.

Draft Model Approaches

- N-gram**: Near-zero latency but low τ (poor approximation).
- Eagle3**: Autoregressive drafter; high τ but $O(K)$ sequential passes—draft latency grows with block size.
- DFlash**: Diffusion-based block drafter—all $K=16$ tokens in **one pass** via non-causal attention. Draft cost is $O(1)$ regardless of K , uniquely suited for TPU’s flat verification profile.

DFlash Architecture

- 4-layer transformer with non-causal block attention; each draft token attends bidirectionally to all other draft tokens.
- Reuses target model’s embedding layer and LM head (no extra params).
- Conditioned on target hidden states from layers [1,9,17,25,33], projected via FC + RMSNorm.
- Trained on only 289K samples; outperforms Eagle3 (1.4M samples).

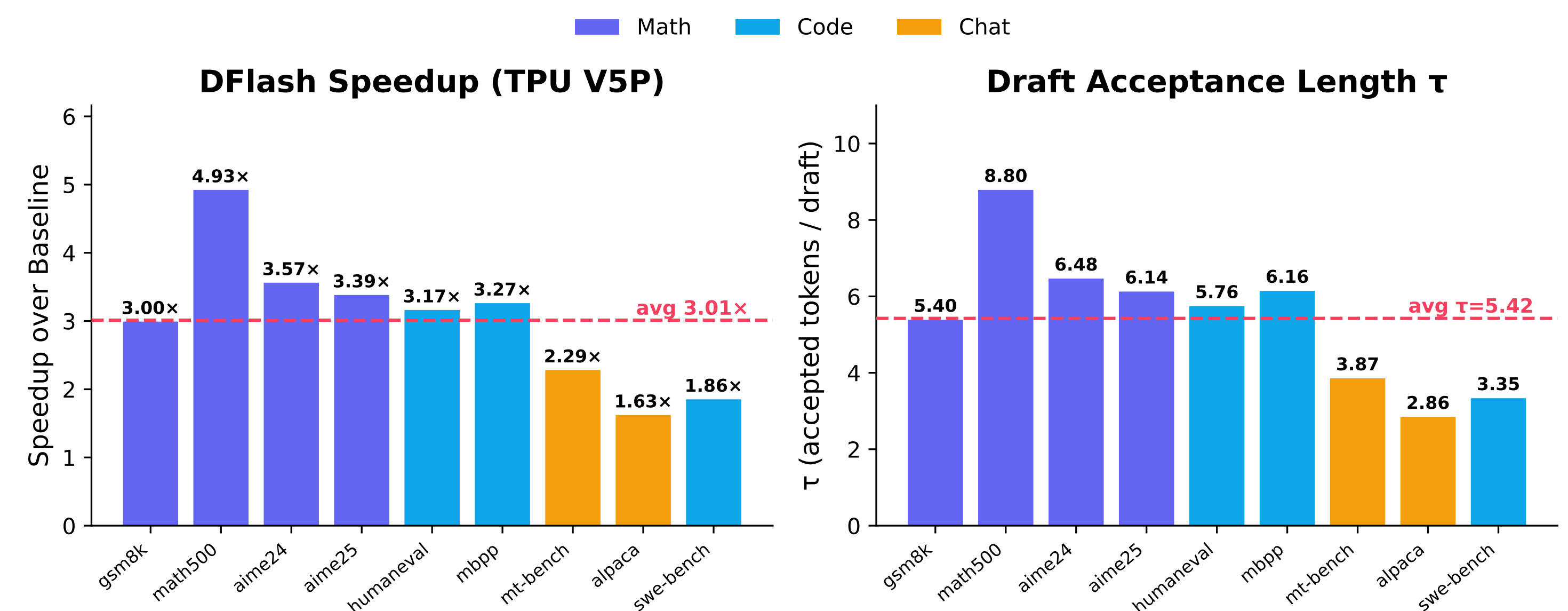
Methods: GPU $\rightarrow$ TPU

- Dual KV cache**: Paged KV (PagedAttention) for target; static JAX KV with `dynamic_update_slice` for draft.
- Non-causal attention**: TPU `flash_attention` with `causal=False` for bidirectional draft-token attention.
- Critical bug fix—sequence-length inflation**: Phantom tokens (~ 15 /step) corrupted RoPE embeddings. Fixing this *nearly doubled performance*:
 τ : $2.49 \rightarrow 4.48$ Speedup: $1.30\times \rightarrow 2.31\times$
- Hidden-state extraction**: Layers [1,9,17,25,33] concatenated and projected via learned FC + RMSNorm.
- Method registration**: New **"dflash"** branch in `tpu_runner.py` with prewarm and cache-reset support.

Zero new vLLM dependencies. Full DFlash contract preserved.

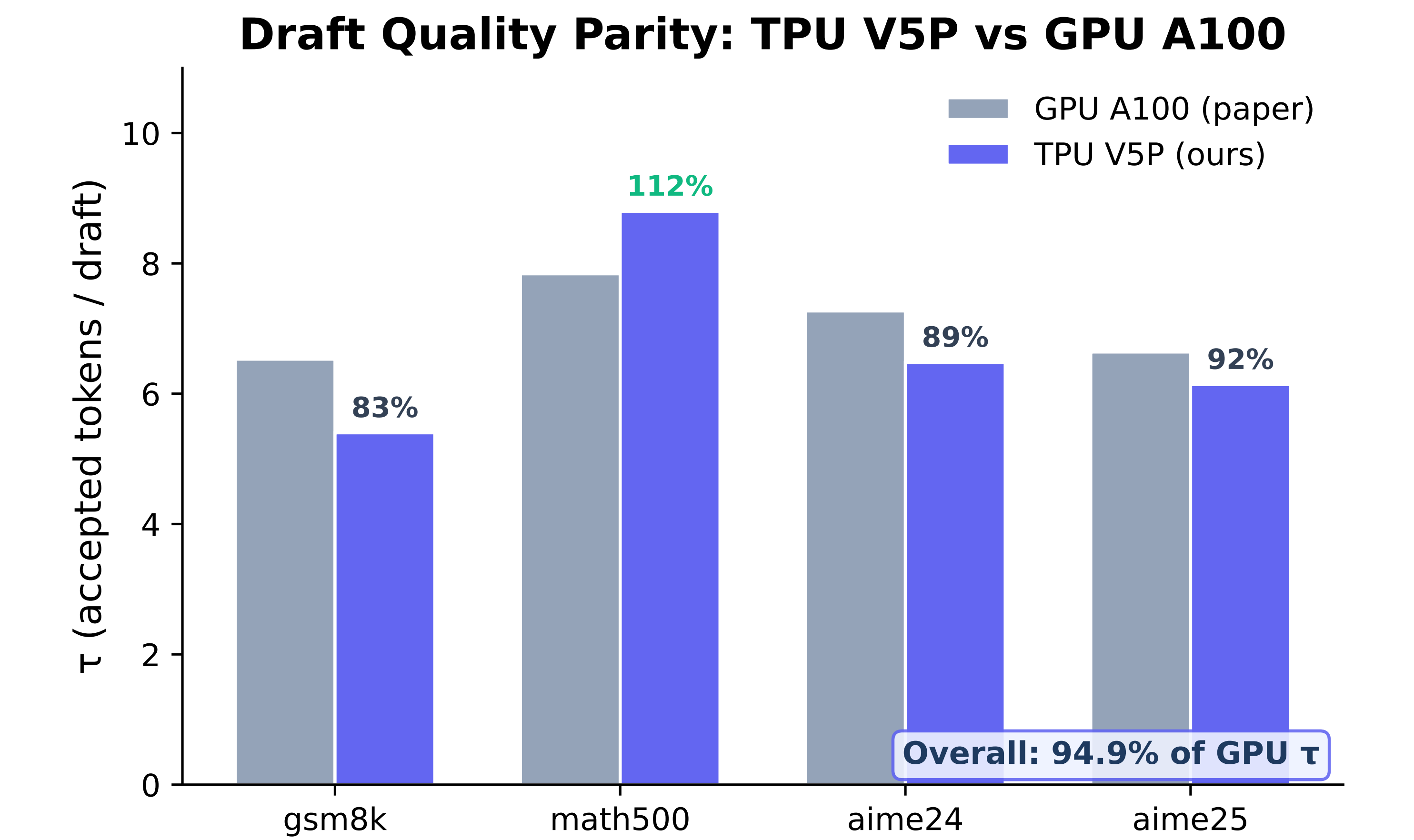
Key Results

$3.01\times$ Standalone $\tau = 5.42$ Accepted/Draft $2.31\times$ Pipeline



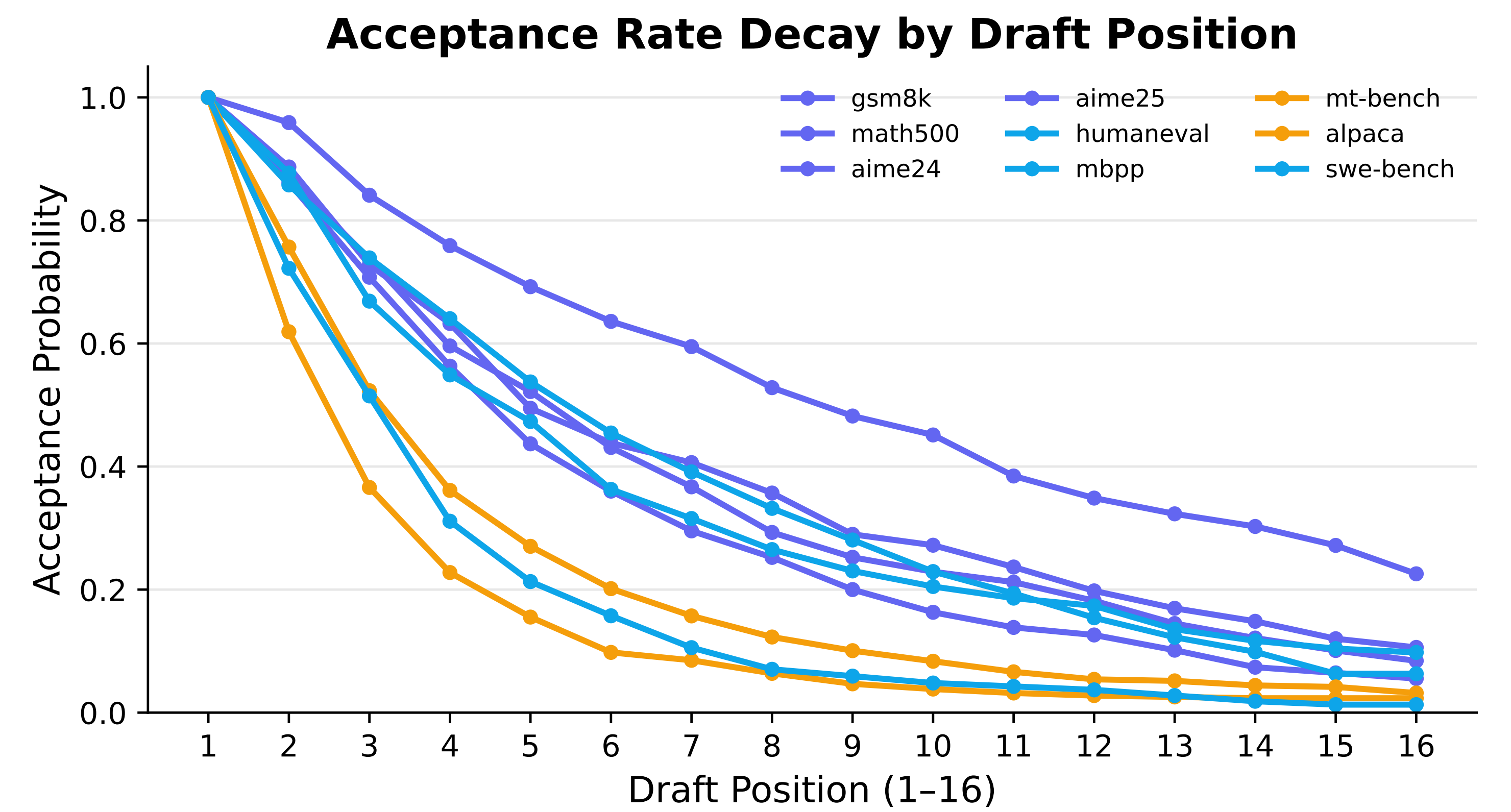
- Setup**: Qwen3-4B + DFlash block-16 on TPU V5P (4 chips), evaluated on **9 benchmarks** in 3 categories.
- Math** (gsm8k, math500, aime24, aime25): **$3.72\times$** avg. speedup, $\tau=6.71$. *Math500 peaks at $4.93\times$ ($\tau=8.80$)*, exceeding GPU paper’s $\tau=7.84$.
- Code** (humaneval, mbpp): **$2.77\times$** avg. speedup, $\tau=5.09$. Structured patterns boost acceptance.
- Chat** (mt-bench, alpaca, swe-bench): **$1.96\times$** avg. speedup, $\tau=3.37$. Higher entropy limits draft accuracy.
- Output quality preserved**: Divergence is numerical (bf16 rounding), not semantic errors.

TPU vs GPU Draft Quality



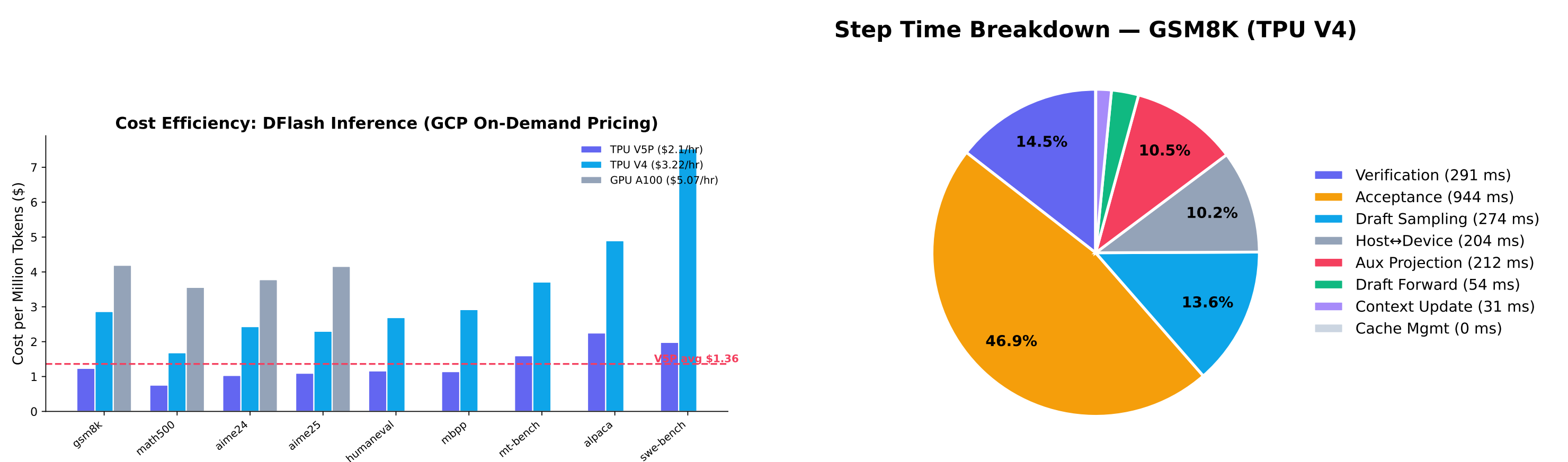
- Draft quality parity**: Our TPU port achieves **94.9%** of GPU A100 τ across the 4 math benchmarks with available GPU data.
- Math500 *exceeds* GPU at **112%**—the only benchmark where TPU surpasses the original GPU implementation.
- Remaining $\sim 5\%$ gap attributed to bf16 precision and minor attention kernel divergence between PyTorch and JAX.

Acceptance Rate by Position



- Math benchmarks maintain $>70\%$ acceptance through positions 8–10, reflecting high predictability of step-by-step reasoning.
- Chat tasks drop below 50% by position 4–5 due to higher output entropy and creative language patterns.
- DFlash’s **non-causal attention** lets each position attend bidirectionally, sustaining quality at later draft positions.
- Even at position 16, math retains $>20\%$ acceptance, justifying wide $K=16$ blocks.

Cost Efficiency & Profiling



- TPU V5P: **\$1.36/M tokens** avg. vs. \$2.08 (V4) and \$2.94 (GPU A100 est.)—the most cost-efficient option.
- Verification (46.9%) and acceptance (14.5%) dominate step time; draft forward is only 2.7% (~ 54 ms).
- Pipeline overhead** (host–device transfer, cache mgmt.) explains the standalone–pipeline τ gap (5.42 vs 4.48).

Conclusion & Future Work

- DFlash on TPU V5P: **$3.01\times$ standalone, $2.31\times$ pipeline** speedup with no quality loss.
- TPU’s flat verification scaling makes it the ideal hardware for speculative decoding with large draft blocks.
- Most cost-efficient inference configuration among all hardware–method combinations tested.

Future Work

- Wider blocks ($K=64, 128$)**: Both draft and verify stay flat on TPU—should scale near-linearly.
- vLLM optimization**: Reduce host–device overhead to close the standalone–pipeline gap.
- LM head fusion**: Two matmuls account for $\sim 30\%$ of step time and can be fused into one kernel.

References & Acknowledgements

- References**
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. vLLM: Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
 - Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up speculative sampling requires rethinking draft model training. In *arXiv preprint arXiv:2503.01840*, 2025.
 - Hao Zhang et al. DFlash: Accelerating autoregressive inference with diffusion-based speculative decoding. *arXiv preprint*, 2025. <https://github.com/z-lab/DFlash>.

Acknowledgements: We thank our mentor Hao Zhang and TA Yiming Zhao for their guidance, and the UCSD HDSI for supporting this DSC180B capstone project.